

Exploratory Data Analysis on Titanic Dataset

(DATA ANALYSIS and VISUALIZATION)

```
In [11]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Optional for better plot style
sns.set_style("whitegrid")

# Load the dataset
df = pd.read_csv("train.csv")

# Display the first few rows
df.head()
```

```
Out[11]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Titanic Dataset - Initial Data Exploration

We have loaded the Titanic dataset to begin our exploratory analysis.

This dataset contains details about passengers on the Titanic and their survival.

```
In [12]: # Check for missing values in the dataset
df.isnull().sum()
```

```
Out[12]: PassengerId      0
Survived      0
Pclass        0
Name          0
Sex           0
Age          177
SibSp         0
Parch         0
Ticket        0
Fare          0
Cabin        687
Embarked      2
dtype: int64
```

Handling Missing Values

First, we checked the missing values across the dataset.

Here is how we handled them:

- Filled missing `Age` with the median of the `Age` column.
- Filled missing `Embarked` with the most frequent value (mode).
- Dropped the `Cabin` column due to high missing values.

```
In [22]: # Fill missing 'Age' with median
df['Age'] = df['Age'].fillna(df['Age'].median())

# Fill missing 'Embarked' with mode
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])

# Check if 'Cabin' column exists before dropping
```

```

if 'Cabin' in df.columns:
    df = df.drop(columns=['Cabin'])

# Check the dataset after handling missing values
df.isnull().sum()

```

```

Out[22]: PassengerId    0
         Survived      0
         Pclass       0
         Name        0
         Sex         0
         Age         0
         SibSp       0
         Parch       0
         Ticket      0
         Fare        0
         Embarked    0
         FamilySize  0
         dtype: int64

```

We handled missing data by:

- Filling missing `Age` values with the median value.
- Filling missing `Embarked` values with the mode.
- Dropping the `Cabin` column due to a large number of missing values.

```

In [14]: # Basic information about the dataset
df.info()

# Statistical summary
df.describe()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PassengerId     891 non-null   int64
1   Survived        891 non-null   int64
2   Pclass          891 non-null   int64
3   Name            891 non-null   object
4   Sex             891 non-null   object
5   Age             891 non-null   float64
6   SibSp           891 non-null   int64
7   Parch           891 non-null   int64
8   Ticket          891 non-null   object
9   Fare            891 non-null   float64
10  Embarked        891 non-null   object
dtypes: float64(2), int64(5), object(4)
memory usage: 76.7+ KB

```

```

Out[14]:

```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	891.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.361582	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	13.019697	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	22.000000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	35.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Dataset Summary

- We have X rows and Y columns.
- The summary statistics give us insight into numerical data like age and fare.
- Next, we'll look at the distribution of key features like `Sex` , `Embarked` , `Pclass` .

```

In [15]: # Display value counts for key categorical features
df['Sex'].value_counts(), df['Embarked'].value_counts(), df['Pclass'].value_counts()

```

```
Out[15]: (Sex
male      577
female    314
Name: count, dtype: int64,
Embarked
S      646
C      168
Q       77
Name: count, dtype: int64,
Pclass
3      491
1      216
2      184
Name: count, dtype: int64)
```

Categorical Data Distribution

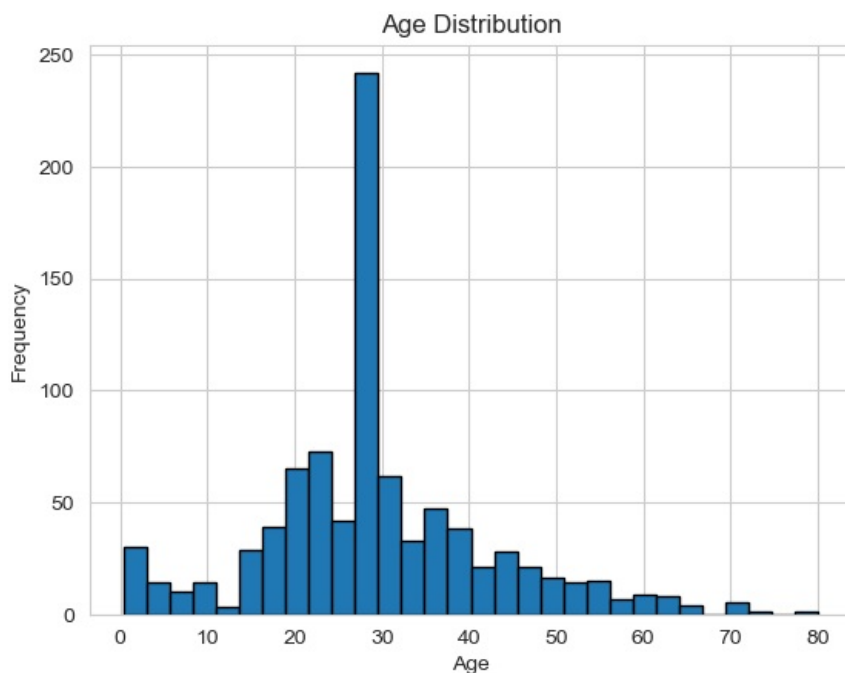
- The `Sex` feature shows more males than females.
- The most common embarkation port is 'S' (Southampton).
- The dataset contains passengers across three classes, with the majority in 3rd class.

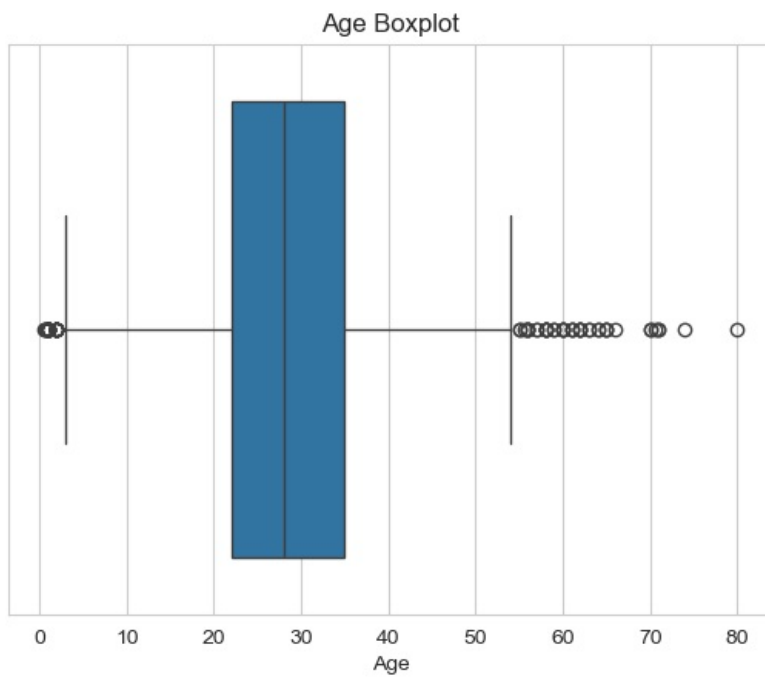
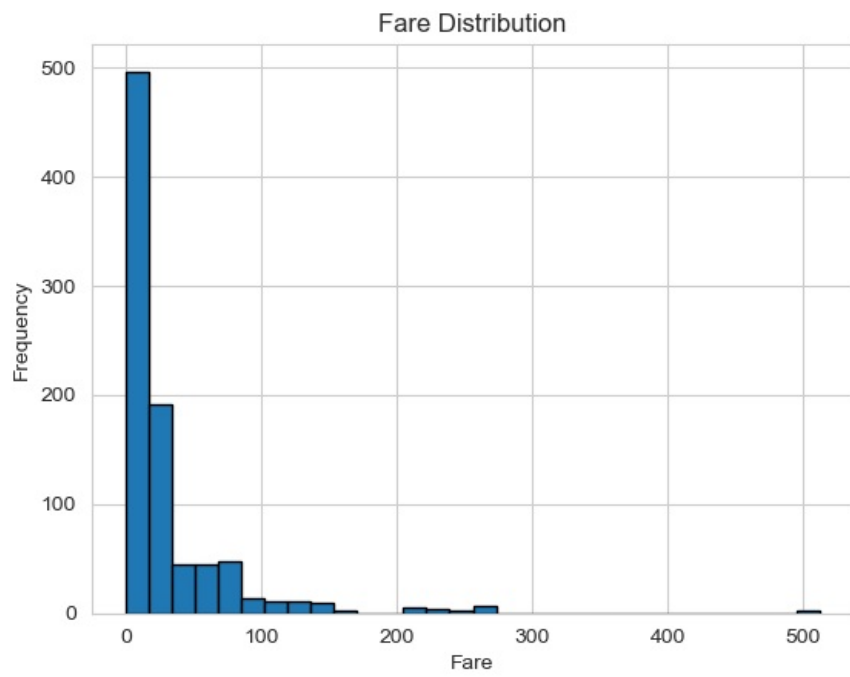
```
In [16]: # Age Distribution Histogram
df['Age'].hist(bins=30, edgecolor='black')
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()

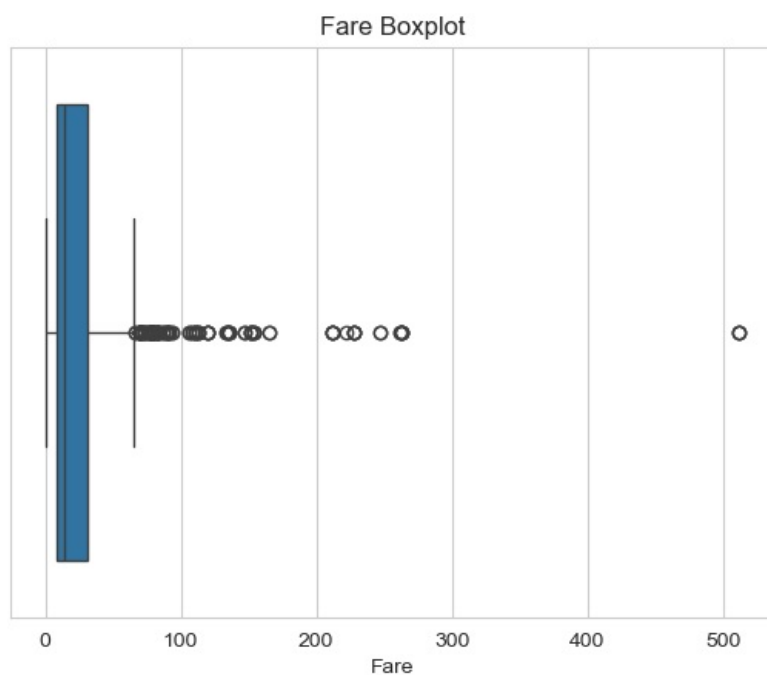
# Fare Distribution Histogram
df['Fare'].hist(bins=30, edgecolor='black')
plt.title('Fare Distribution')
plt.xlabel('Fare')
plt.ylabel('Frequency')
plt.show()

# Boxplot for Age
sns.boxplot(x=df['Age'])
plt.title('Age Boxplot')
plt.show()

# Boxplot for Fare
sns.boxplot(x=df['Fare'])
plt.title('Fare Boxplot')
plt.show()
```







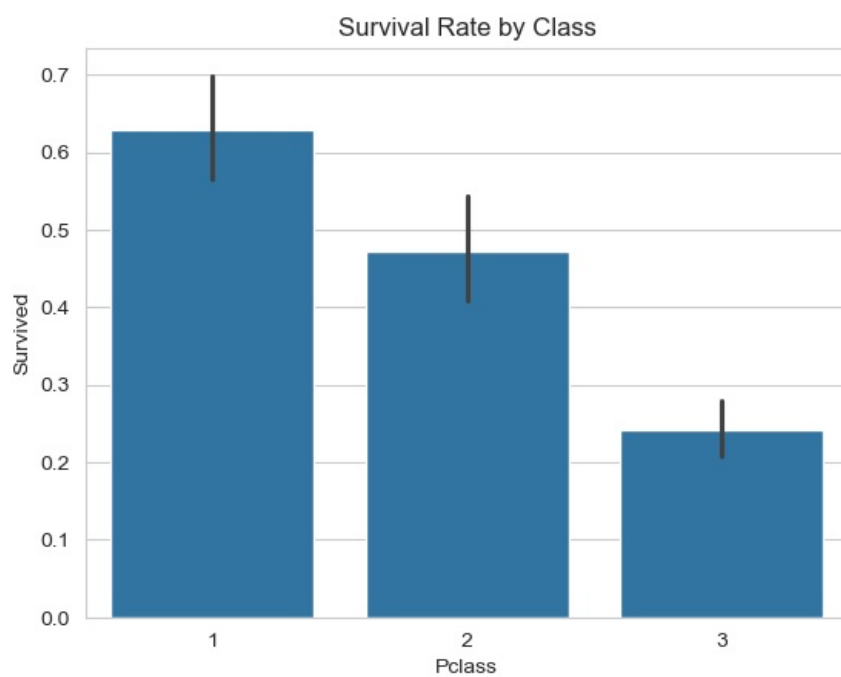
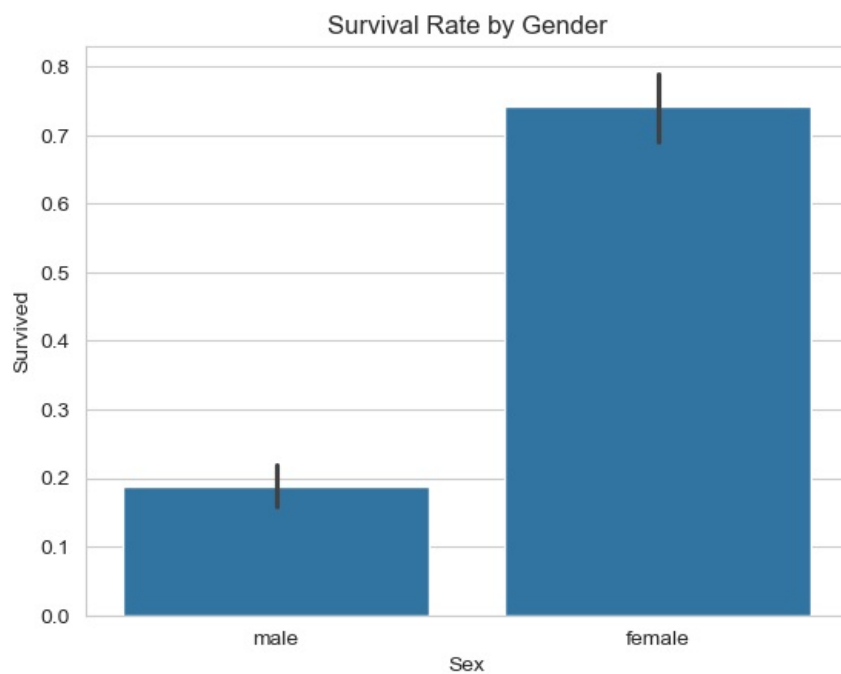
Visualizing Data Distributions

- Age and Fare distributions reveal that:
 - Age is mostly between 20 and 40 years.
 - Fare is right-skewed, with some passengers paying very high fares.
 - Outliers are present in both Age and Fare.

Boxplots show a few extreme outliers in both age and fare.

```
In [17]: # Survival Rate by Gender
sns.barplot(x='Sex', y='Survived', data=df)
plt.title('Survival Rate by Gender')
plt.show()

# Survival Rate by Class
sns.barplot(x='Pclass', y='Survived', data=df)
plt.title('Survival Rate by Class')
plt.show()
```

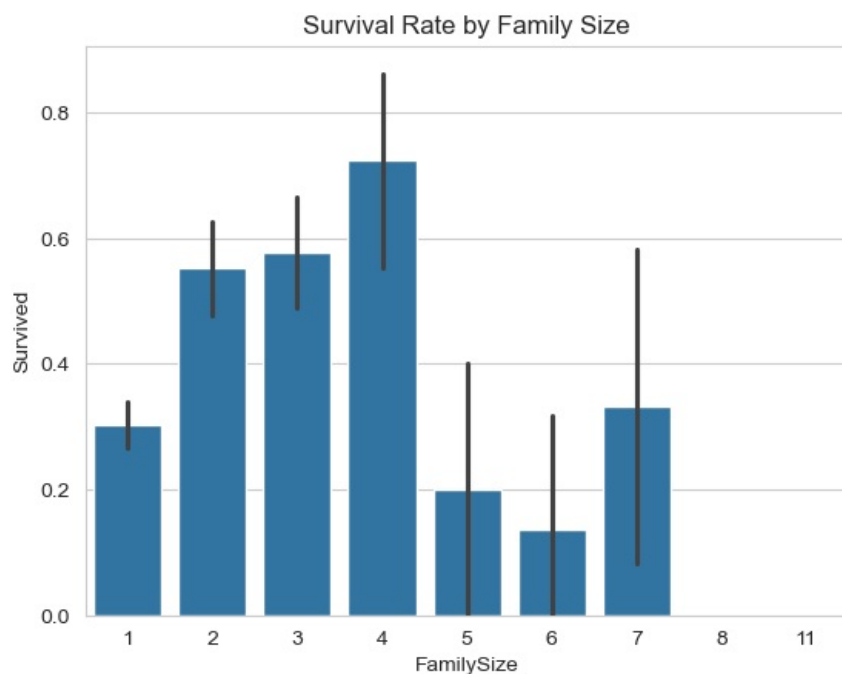


Survival Rates by Gender and Class

- Females had a significantly higher survival rate compared to males.
- First-class passengers had the highest survival rate, while 3rd class passengers had the lowest survival.

```
In [18]: # Creating a new feature: Family Size
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1

# Visualizing Survival by Family Size
sns.barplot(x='FamilySize', y='Survived', data=df)
plt.title('Survival Rate by Family Size')
plt.show()
```



Feature Engineering – Family Size

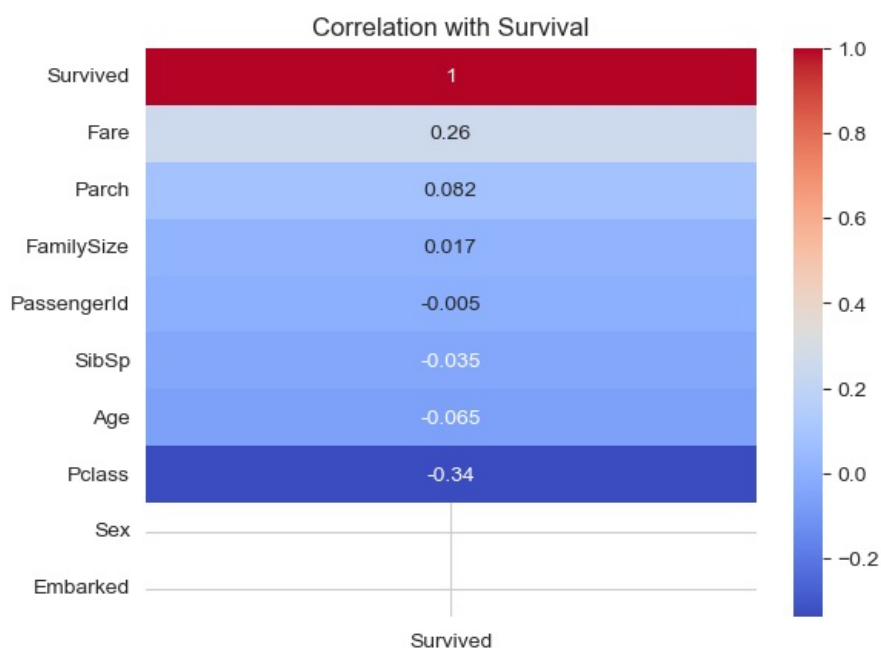
- A new feature `FamilySize` was created by summing `SibSp` and `Parch` and adding 1.
- Smaller families (`FamilySize < 3`) showed better survival rates, while larger families had lower survival chances.

```
In [23]: # Convert categorical columns to numeric
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
df['Embarked'] = df['Embarked'].map({'S': 0, 'C': 1, 'Q': 2})

# Select only numeric columns for correlation
numeric_columns = df.select_dtypes(include=['float64', 'int64']).columns

# Correlation with target 'Survived'
corr = df[numeric_columns].corr()

# Plot the correlation heatmap
sns.heatmap(corr[['Survived']].sort_values(by='Survived', ascending=False), annot=True, cmap='coolwarm')
plt.title('Correlation with Survival')
plt.show()
```



Correlation with Survival

- The highest positive correlations with survival were `Sex` (female) and `Pclass` (1st class).

- Fare also shows a moderate positive correlation with survival.
- SibSp and FamilySize are weakly negatively correlated with survival.

Conclusion

- We have cleaned and explored the Titanic dataset to uncover key patterns.
- Gender, class, and fare are the most important features influencing survival.
- Family size can be an interesting feature for predictive modeling.

In []:

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js